

RELATÓRIO DO PROJETO JACKUT

Disciplina: Programação 2

Projeto: Jackut – Milestone 1 e 2

Linguagem: Java

Aluno: Artur Ferreira Marques da Silva

Professor: Mario Hozano Lucas de Souza

Período: 2026.1

1. Introdução

Este relatório apresenta a versão final do projeto Jackut desenvolvida para a disciplina de Programação 2. O sistema consiste em uma rede de relacionamentos inspirada no Orkut, implementada em Java e validada por testes de aceitação executados com a ferramenta EasyAccept.

No Milestone 01, o projeto contempla a base inicial da aplicação, incluindo criação de usuários, abertura de sessões, edição e consulta de perfil, convites de amizade, efetivação de amizades, envio de recados e leitura de recados. No Milestone 02, o sistema é expandido para contemplar comunidades, participação em comunidades, mensagens para comunidades, novos tipos de relacionamento social, como fã, ídolo, paquera e inimigo, além da remoção completa de contas do sistema.

Além da implementação das novas User Stories, a versão final também apresenta uma refatoração arquitetural importante em relação ao Milestone 01. A classe SistemaJackut, que antes concentrava a maior parte das regras de negócio, deixa de existir. Suas responsabilidades passam a ser distribuídas entre serviços especializados, como UsuarioService, SessaoService, PerfilService, AmizadeService, ComunidadeService, MensagemService, IdoloService, PaqueraService, InimigoService, RecadoService e RemocaoUsuarioService.

Essa mudança reduz o acoplamento, aumenta a coesão das classes e corrige uma fragilidade arquitetural relevante: a existência de uma classe central com características de God Class. A classe Facade permanece como ponto de entrada público usado pelos testes de aceitação, mas passa a atuar apenas como fachada real, delegando as operações para os serviços especializados sem concentrar regras de negócio.

O projeto final também evolui no tratamento de exceções. Em vez de usar uma única exceção genérica para todas as regras de negócio, o sistema passa a possuir uma hierarquia de exceções específicas, como LoginInvalidoException, UsuarioNaoCadastradoException, ComunidadeNaoExisteException, SemRecadosException, SemMensagensException, IdoloJaExistenteException e FuncaoInvalidaException. Essa decisão melhora a legibilidade do código e torna mais clara a semântica dos erros do domínio.

Este relatório descreve a arquitetura final do sistema, seus principais componentes, as interações entre as classes, os conceitos de programação orientada a objetos aplicados, os padrões e princípios arquiteturais utilizados, a persistência de dados, a compatibilidade de execução e os testes realizados.

2. Descrição geral do design arquitetural do sistema

O projeto Jackut é organizado de forma modular, separando entidades de domínio, serviços de negócio, contratos, persistência, exceções, utilitários e a fachada pública do sistema. Essa divisão busca atender ao princípio da responsabilidade única, evitando que uma mesma classe concentre dados, validações, regras de negócio, formatação, persistência e controle de fluxo.

A estrutura principal do sistema é dividida em:

- **modelos:** contém as entidades e objetos de apoio do domínio, como `Usuario`, `Sessao`, `Comunidade`, `Recado`, `MensagemComunidade`, `PerfilUsuario`, `AmizadesUsuario`, `ParticipacoesUsuario`, `RelacionamentosUsuario` e `CaixaEntrada`;
- **servicos:** contém as classes responsáveis pelas regras de negócio, como `UsuarioService`, `SessaoService`, `PerfilService`, `AmizadeService`, `ComunidadeService`, `MensagemService`, `IdoloService`, `PaqueraService`, `InimigoService`, `RecadoService`, `BloqueioRelacionamentoService`, `RemocaoUsuarioService` e `ServicoPersistencia`;
- **contratos:** contém interfaces que definem contratos de comportamento, como `Identificavel` e `Persistidor`;
- **dados:** contém `DadosJackut`, classe responsável por centralizar as estruturas de dados persistentes do sistema;
- **excecoes:** contém a hierarquia de exceções do domínio;
- **util:** contém classes utilitárias, como `FormatadorLista`;
- **Facade:** representa o ponto de entrada público usado pelos testes de aceitação;
- **Main:** executa os arquivos de teste por meio do `EasyAccept`.

A arquitetura final utiliza a classe `Facade` como camada de entrada. Os testes de aceitação chamam diretamente seus métodos públicos. A `Facade`, por sua vez, delega cada operação ao serviço especializado correspondente. Por exemplo, operações relacionadas a usuários são encaminhadas para `UsuarioService`; operações de sessão são encaminhadas para `SessaoService`; operações de comunidades são encaminhadas para `ComunidadeService`; operações de paquera são encaminhadas para `PaqueraService`.

Essa separação evita que a Facade contenha lógica de negócio. Ela conhece os serviços, mas não implementa diretamente as regras. A validação de senha, por exemplo, pertence a `UsuarioService`; a validação de sessão pertence a `SessaoService`; a regra de bloqueio por inimizade pertence a `BloqueioRelacionamentoService`; a regra de recado automático por paquera recíproca pertence a `PaqueraService`.

A classe `DadosJackut` atua como repositório central do estado persistível da aplicação. Ela mantém os mapas de usuários, sessões e comunidades. Embora as sessões sejam mantidas durante a execução, elas são reiniciadas após carregamento da persistência, pois representam estado temporário e não uma informação permanente do domínio.

A classe `Usuario` é uma entidade central, mas não concentra mais diretamente todos os detalhes de perfil, amizades, comunidades, relacionamentos e mensagens. Esses dados são distribuídos por composição em classes auxiliares: `PerfilUsuario`, `AmizadesUsuario`, `ParticipacoesUsuario`, `RelacionamentosUsuario` e `CaixaEntrada`. Essa decisão reduz o tamanho conceitual de `Usuario` e evita que a entidade se transforme em uma classe excessivamente grande e pouco coesa.

A persistência é isolada pela interface `Persistidor` e pela implementação `ServicoPersistencia`. Com isso, o restante do sistema depende do contrato de persistência e não diretamente da implementação concreta baseada em serialização. Essa decisão favorece extensibilidade, pois uma futura versão poderia persistir os dados em banco de dados ou outro formato, desde que respeite o contrato definido por `Persistidor`.

A organização final apresenta alta coesão interna. Cada serviço possui um eixo de responsabilidade claro. `UsuarioService` conhece regras de cadastro e busca de usuários. `SessaoService` conhece autenticação e validação de sessão. `ComunidadeService` conhece criação, consulta e participação em comunidades. `MensagemService` conhece envio e leitura de mensagens de comunidades. Os serviços `IdoloService`, `PaqueraService` e `InimigoService` separam os diferentes tipos de relacionamento social, evitando que todos fiquem concentrados em uma única classe.

Essa arquitetura também diminui o acoplamento entre funcionalidades. A regra de bloqueio por inimizade, por exemplo, é usada por amizade, recado, ídolo e paquera. Em vez de repetir essa validação em cada serviço, ela é centralizada em `BloqueioRelacionamentoService`. Dessa forma, caso a regra de bloqueio seja alterada em uma etapa futura, a alteração fica localizada em um único ponto do sistema.

3. Diagrama de classes

Ilustração 1 - Diagrama simplificado da arquitetura geral do Jackut

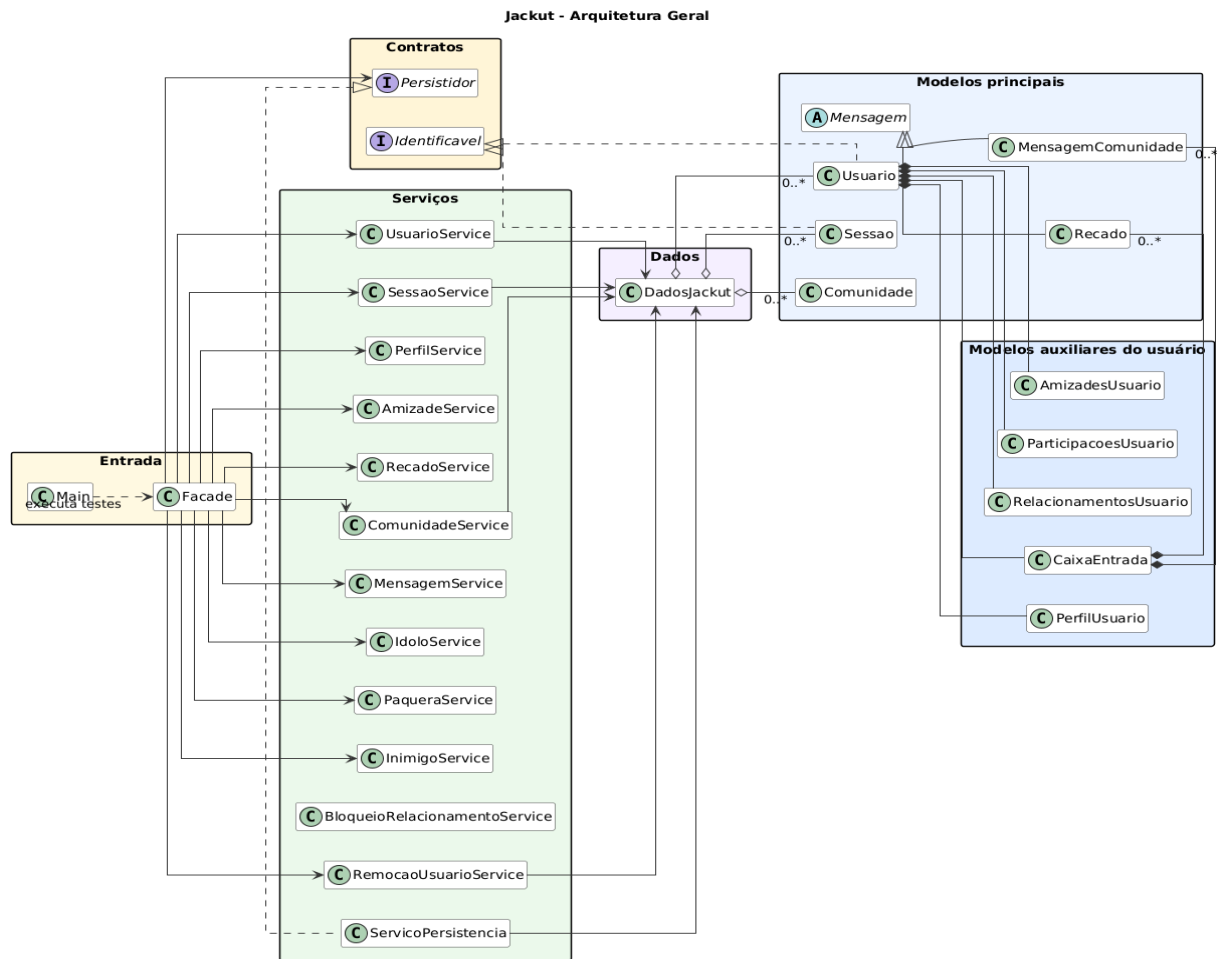


Ilustração 2 - Diagrama simplificado de comunidades, mensagens e remoção de conta

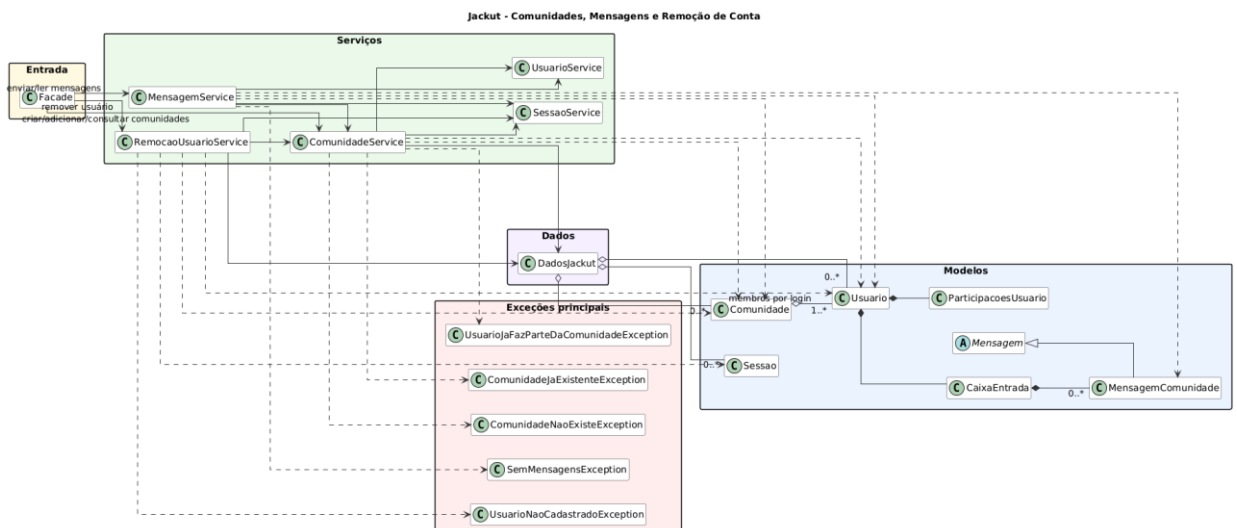
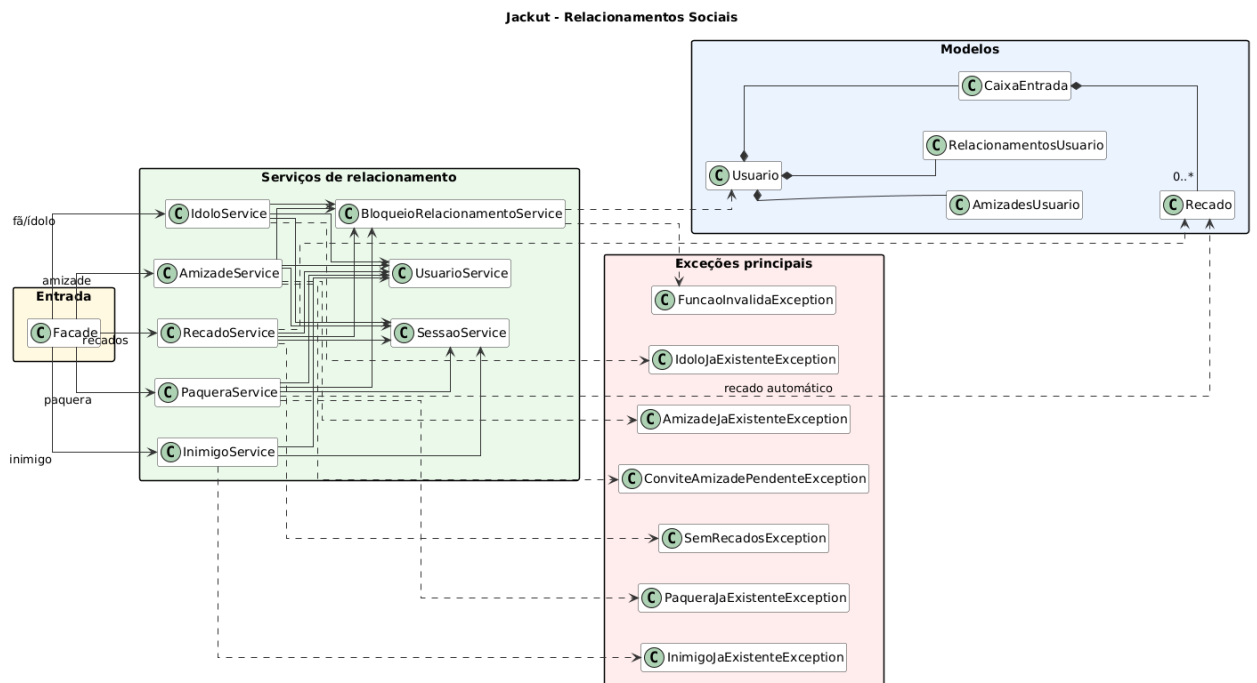


Ilustração 3 - Diagrama simplificado dos relacionamentos sociais



4. Principais componentes e suas interações

4.1 Classe Main

A classe Main representa o ponto de execução dos testes de aceitação. Ela não contém regra de negócio do Jackut. Seu papel é localizar a pasta de testes e executar os arquivos de teste por meio da ferramenta EasyAccept.

Seus principais elementos são o nome da classe Facade usada como ponto de entrada e os métodos auxiliares de localização e execução dos testes. A existência dessa classe facilita a validação do projeto em ambientes diferentes, pois evita dependência rígida de caminhos absolutos no computador do desenvolvedor.

Sem a classe Main, ainda seria possível executar os testes manualmente, mas a validação do projeto ficaria menos prática e mais sujeita a erros de configuração.

4.2 Classe Facade

A classe Facade é a fachada pública do sistema. Ela expõe todos os métodos esperados pelos testes de aceitação, como `criarUsuario`, `abrirSessao`, `editarPerfil`, `adicionarAmigo`, `enviarRecado`, `criarComunidade`, `adicionarComunidade`, `enviarMensagem`, `adicionarIdolo`, `adicionarPaquera`, `adicionarInimigo` e `removerUsuario`.

Seus principais atributos são referências para DadosJackut, Persistidor e os serviços especializados. Ela inicializa esses serviços no método `inicializarServicos` e delega cada chamada pública para o serviço correspondente.

A justificativa da existência da Facade está no padrão de projeto Facade. O EasyAccept precisa de um ponto único de acesso ao sistema, mas não deve conhecer a estrutura interna de pacotes, serviços e modelos. A Facade resolve esse problema ao fornecer uma interface estável e simples para o mundo externo.

Na versão final, a Facade não atua como God Class, pois não implementa diretamente as regras de negócio. Ela apenas coordena a inicialização dos serviços e encaminha chamadas. Essa diferença é importante porque preserva a compatibilidade com os testes sem comprometer a separação de responsabilidades.

4.3 Interface Identificavel

A interface `Identificavel` define o contrato para classes que possuem um identificador textual. Seu principal método é `getId`.

Ela é implementada por `Usuario` e `Sessao`. A existência dessa interface substitui a antiga abordagem baseada em herança estrutural com uma classe `Entidade`. Essa decisão é mais adequada porque o sistema precisa apenas de um contrato comum de identificação, e não de uma superclasse com estado compartilhado.

O uso de `Identificavel` evita herança artificial. `Usuario` e `Sessao` são conceitos diferentes no domínio, mas ambos podem ser tratados como objetos identificáveis. Dessa forma, a abstração fica mais leve, flexível e coerente.

4.4 Interface Persistidor

A interface `Persistidor` define o contrato de persistência do sistema. Seus métodos principais são `salvar`, `carregar` e `limpar`.

Ela permite que a Facade e os demais componentes trabalhem com uma abstração de persistência, e não com uma implementação concreta. Atualmente, `ServicoPersistencia` implementa esse contrato usando serialização Java.

A existência dessa interface reduz acoplamento e favorece extensibilidade. Se em um próximo milestone fosse necessário salvar os dados em XML, JSON ou banco de dados, uma nova classe poderia implementar `Persistidor` sem alterar o contrato usado pela Facade.

4.5 Classe DadosJackut

A classe `DadosJackut` centraliza o estado principal do sistema. Seus principais atributos são os mapas de usuários, sessões e comunidades, além do contador de sessão.

Ela possui métodos para acessar essas estruturas, gerar identificadores de sessão, reiniciar sessões e limpar os dados. Sua existência evita que cada serviço mantenha seu próprio armazenamento isolado, o que poderia gerar inconsistência.

DadosJackut funciona como objeto agregador do estado persistível. UsuarioService, SessaoService, ComunidadeService e RemocaoUsuarioService utilizam essa classe para acessar ou modificar informações compartilhadas. Essa centralização é importante para a persistência, pois ServicoPersistencia salva e carrega um único objeto contendo o estado relevante do sistema.

4.6 Classe Usuario

A classe Usuario representa o usuário cadastrado no Jackut. Seus principais atributos são login, senha, perfil, amigos, participacoes, relacionamentos e caixaEntrada.

O login identifica o usuário dentro do sistema, enquanto a senha é usada para autenticação. Os demais atributos são objetos de apoio responsáveis por partes específicas do estado do usuário. PerfilUsuario armazena atributos de perfil; AmizadesUsuario armazena amigos e convites; ParticipacoesUsuario armazena comunidades; RelacionamentosUsuario armazena ídolos, fãs, paqueras e inimigos; CaixaEntrada armazena recados e mensagens.

A classe oferece métodos de alto nível, como editarPerfil, ehAmigo, adicionarComunidade, adicionarIdolo, adicionarPaquera, adicionarInimigo, receberRecado e receberMensagem. Internamente, esses métodos delegam a manipulação do estado para os objetos compostos.

A justificativa dessa estrutura está no princípio da composição. Em vez de concentrar todas as listas e regras auxiliares diretamente em Usuario, a classe passa a ser uma entidade agregadora. Isso melhora a coesão e reduz o risco de Usuario se tornar uma God Class.

4.7 Classe PerfilUsuario

A classe PerfilUsuario representa os dados de perfil de um usuário. Seu principal atributo é um mapa de atributos textuais.

Ela possui métodos para editar um atributo, verificar se determinado atributo existe e recuperar o valor de um atributo. O uso de mapa é adequado porque os testes permitem atributos variados, como nome, descrição, estado civil, aniversário e cidade natal.

Sem essa classe, Usuario precisaria manipular diretamente o mapa de perfil, aumentando sua responsabilidade. Ao separar PerfilUsuario, o sistema atribui a responsabilidade ao objeto que contém os dados necessários, seguindo o princípio Expert.

4.8 Classe AmizadesUsuario

A classe AmizadesUsuario agrupa o estado relacionado a amizades de um usuário. Seus principais atributos são a lista de amigos e a lista de convites enviados.

Ela possui métodos para verificar amizade, adicionar amigo efetivado, adicionar convite enviado, remover convite enviado, verificar convite pendente e retornar a lista de amigos.

A existência dessa classe melhora a modelagem porque amizade e convite possuem regras próprias e não são apenas listas genéricas. Sem ela, Usuario precisaria conhecer diretamente todos os detalhes de convites e amizades, o que aumentaria o acoplamento interno da entidade.

4.9 Classe ParticipacoesUsuario

A classe ParticipacoesUsuario armazena as comunidades das quais um usuário participa. Seu principal atributo é a lista de nomes de comunidades.

Ela possui métodos para adicionar comunidade, remover comunidade, verificar participação e retornar comunidades. Essa classe separa a responsabilidade de participação em comunidades da entidade Usuario.

Essa decisão é importante para a extensibilidade. Caso futuras funcionalidades adicionem papéis em comunidades, como moderador ou administrador, essa classe seria o local natural para evoluir o conceito de participação sem alterar diretamente toda a estrutura de Usuario.

4.10 Classe RelacionamentosUsuario

A classe RelacionamentosUsuario concentra os relacionamentos sociais que não são amizades. Seus principais atributos são as listas de ídolos, fãs, paqueras e inimigos.

Ela possui métodos para adicionar ídolo, verificar se o usuário é fã de alguém, adicionar fã, adicionar paquera, verificar paquera, adicionar inimigo, verificar inimigo e remover referências a um usuário removido.

A existência dessa classe evita que Usuario mantenha diretamente todas as listas de relacionamento social. Além disso, ela deixa claro que amizade é um relacionamento tratado separadamente, enquanto fã, ídolo, paquera e inimigo pertencem a outro conjunto de relações.

4.11 Classe CaixaEntrada

A classe CaixaEntrada representa a caixa de entrada de um usuário. Seus principais atributos são uma fila de recados e uma fila de mensagens de comunidade.

Ela possui métodos para receber recados, verificar recados pendentes, remover o primeiro recado, receber mensagens, verificar mensagens pendentes, remover a primeira mensagem, remover recados de um remetente e remover mensagens de um remetente.

A escolha de LinkedList é coerente porque recados e mensagens precisam ser lidos em ordem de chegada. A classe evita que Usuario manipule diretamente estruturas de fila e permite que a lógica de caixa de entrada evolua de forma isolada.

4.12 Classe Sessao

A classe Sessao representa uma sessão aberta por um usuário autenticado. Seus principais atributos são id e loginUsuario.

Ela implementa Identificavel e oferece métodos para recuperar o identificador da sessão e o login do usuário associado. Sua existência permite que operações privadas sejam executadas a partir de uma sessão válida, e não apenas de um login informado diretamente.

Essa modelagem aproxima o sistema de um fluxo real de autenticação. Operações como editarPerfil, adicionarAmigo, enviarRecado e criarComunidade dependem de uma sessão válida para identificar o usuário ativo.

4.13 Classe Comunidade

A classe Comunidade representa uma comunidade criada dentro do Jackut. Seus principais atributos são nome, descricao, dono e membros.

Ela possui métodos para recuperar nome, descrição e dono, adicionar membro, remover membro, verificar se um membro participa da comunidade e retornar a lista de membros.

A classe é uma entidade forte do domínio porque comunidades possuem identidade própria e continuam existindo independentemente da sessão do usuário. O nome funciona como chave principal. A decisão de representar Comunidade como classe própria facilita expansões futuras, como fóruns, tópicos, moderação ou regras de privacidade.

4.14 Classe Mensagem

A classe Mensagem é uma classe abstrata que representa uma mensagem genérica enviada no sistema. Seus principais atributos são remetente e texto.

Ela é abstrata porque o sistema não instancia uma mensagem genérica diretamente. Existem tipos concretos de mensagem, como Recado e MensagemComunidade. Essa estrutura evita duplicação dos atributos comuns e cria uma abstração coerente para objetos que possuem remetente e texto.

A existência de Mensagem favorece o uso de herança com propósito real. Recado e MensagemComunidade compartilham características comuns, mas representam conceitos específicos dentro do domínio.

4.15 Classe Recado

A classe Recado representa uma mensagem enviada diretamente de um usuário para outro. Ela herda de Mensagem e reutiliza os atributos remetente e texto.

Embora os testes leiam apenas o texto do recado, armazenar o remetente é importante para a remoção de conta. Quando um usuário é removido, os recados enviados por ele também precisam ser removidos das caixas de entrada de outros usuários.

Essa decisão demonstra que Recado não é apenas uma String. Ele possui identidade conceitual no domínio e pode evoluir futuramente com data, status de leitura ou histórico.

4.16 Classe MensagemComunidade

A classe MensagemComunidade representa uma mensagem enviada para uma comunidade. Ela herda de Mensagem e acrescenta o atributo comunidade.

Esse atributo identifica a comunidade à qual a mensagem pertence. A classe é necessária porque mensagens de comunidade possuem um contexto diferente dos recados diretos entre usuários.

Sem MensagemComunidade, o sistema teria que tratar mensagens de comunidade como strings soltas, dificultando futuras extensões e a remoção de mensagens enviadas por usuários removidos.

4.17 Classe UsuarioService

A classe UsuarioService é responsável pelas regras de cadastro e busca de usuários. Seu principal atributo é DadosJackut.

Ela possui métodos para criar usuário, buscar usuário e verificar existência de usuário. Também contém validações privadas para login e senha.

A existência desse serviço separa a regra de cadastro da entidade Usuario. Usuario representa o usuário, mas não deve ser responsável por decidir se um novo login pode ser cadastrado ou se uma senha é válida. Essa responsabilidade pertence ao serviço que conhece o conjunto de usuários cadastrados.

4.18 Classe SessaoService

A classe SessaoService é responsável por autenticação e validação de sessões. Seus principais atributos são DadosJackut e UsuarioService.

Ela possui métodos para abrir sessão e buscar o usuário associado a uma sessão. Ao abrir sessão, o serviço valida login e senha e cria um objeto Sessao com identificador gerado por DadosJackut.

Essa classe é essencial para separar autenticação das demais regras. Outros serviços podem depender de SessaoService para obter o usuário ativo sem duplicar a lógica de validação de sessão.

4.19 Classe PerfilService

A classe PerfilService gerencia operações de perfil. Seus principais atributos são UsuarioService e SessaoService.

Ela possui métodos para consultar atributo de usuário e editar perfil. A consulta por login usa UsuarioService, enquanto a edição usa SessaoService para garantir que a operação seja feita por usuário autenticado.

Essa separação mantém a entidade Usuario simples e evita que a Facade precise conhecer detalhes de validação de atributo. A regra de atributo não preenchido fica localizada nesse serviço.

4.20 Classe AmizadeService

A classe AmizadeService é responsável pelas regras de amizade. Seus principais atributos são UsuarioService, SessaoService e BloqueioRelacionamentoService.

Ela possui métodos para adicionar amigo, verificar amizade e listar amigos. O método adicionarAmigo trata três situações principais: tentativa de autoamizade, amizade já existente e convite pendente. Quando existe convite recíproco, a amizade é efetivada.

A existência desse serviço é importante porque amizade possui comportamento próprio. A lógica de convite e efetivação não deve ficar em Usuario, pois depende de dois usuários e do estado global do sistema. Também não deve ficar na Facade, pois é regra de negócio.

4.21 Classe RecadoService

A classe RecadoService gerencia envio e leitura de recados. Seus principais atributos são UsuarioService, SessaoService e BloqueioRelacionamentoService.

Ela possui métodos para enviar recado e ler recado. Ao enviar um recado, o serviço valida a sessão do remetente, busca o destinatário, verifica bloqueio por inimizade, impede autoenvio e adiciona um objeto Recado à caixa de entrada do destinatário.

Essa classe existe para separar a lógica de comunicação direta entre usuários. Sem ela, Usuario ou Facade precisariam conhecer as regras de validação e entrega de recados, aumentando o acoplamento.

4.22 Classe ComunidadeService

A classe ComunidadeService gerencia criação, consulta, participação e remoção de comunidades. Seus principais atributos são DadosJackut, UsuarioService e SessaoService.

Ela possui métodos para criar comunidade, consultar descrição, consultar dono, consultar membros, adicionar usuário a uma comunidade, listar comunidades de um usuário, buscar comunidade, remover comunidade e recuperar comunidades de um dono.

Essa classe é um exemplo claro de alta coesão. Todas as operações relacionadas à estrutura de comunidades ficam centralizadas nela. A entidade Comunidade armazena os dados, mas o serviço conhece as regras de criação, duplicidade, participação e remoção.

4.23 Classe MensagemService

A classe MensagemService é responsável por mensagens enviadas para comunidades. Seus principais atributos são UsuarioService, SessaoService e ComunidadeService.

Ela possui métodos para enviar mensagem e ler mensagem. Ao enviar mensagem, o serviço valida a sessão do remetente, busca a comunidade e entrega uma MensagemComunidade para cada membro da comunidade. Ao ler mensagem, remove a primeira mensagem pendente da caixa de entrada do usuário.

A existência desse serviço separa mensagens comunitárias de recados diretos. Essa distinção é importante porque as regras de entrega são diferentes: recado vai para um usuário específico, enquanto mensagem de comunidade vai para todos os membros da comunidade.

4.24 Classe IdoloService

A classe IdoloService gerencia a relação entre fã e ídolo. Seus principais atributos são UsuarioService, SessaoService e BloqueioRelacionamentoService.

Ela possui métodos para adicionar ídolo, verificar se um usuário é fã de outro e listar fãs de um usuário. Ao adicionar ídolo, o serviço valida a sessão, busca o ídolo, verifica bloqueio por inimizade, impede que o usuário seja fã de si mesmo e evita duplicidade.

A separação em IdoloService evita que todas as relações sociais fiquem em um único serviço genérico. Isso aumenta a coesão e facilita manutenção futura.

4.25 Classe PaqueraService

A classe PaqueraService gerencia relações de paquera. Seus principais atributos são UsuarioService, SessaoService e BloqueioRelacionamentoService.

Ela possui métodos para adicionar paquera, verificar paquera, listar paqueras e enviar recados automáticos quando existe paquera recíproca. Quando dois usuários se adicionam mutuamente como paquera, o serviço envia recados automáticos para ambos.

Essa classe é necessária porque paquera possui uma regra específica que não existe em outros relacionamentos: a notificação automática de reciprocidade. Separar essa regra evita que IdoloService ou InimigoService conheçam detalhes que não fazem parte de suas responsabilidades.

4.26 Classe InimigoService

A classe InimigoService gerencia a relação de inimizade. Seus principais atributos são UsuarioService e SessaoService.

Ela possui método para adicionar inimigo, validando a sessão, a existência do usuário alvo, a tentativa de auto inimizade e a duplicidade de inimigo.

A relação de inimizade possui impacto em outras operações do sistema, mas a validação de bloqueio é isolada em BloqueioRelacionamentoService. Assim, InimigoService apenas registra a relação, enquanto o bloqueio de ações é aplicado pelos serviços que realizam operações contra outro usuário.

4.27 Classe BloqueioRelacionamentoService

A classe BloqueioRelacionamentoService é responsável por validar se uma ação está bloqueada por inimizade.

Seu principal método é validarBloqueioPorInimizade, que recebe o usuário que tenta executar a ação e o alvo da ação. Se o alvo possui o usuário atual como inimigo, o serviço lança FuncaoInvalidaException.

Essa classe evita duplicação de regra. AmizadeService, RecadoService, IdoloService e PaqueraService precisam aplicar a mesma validação. Centralizar essa regra reduz acoplamento e melhora a manutenibilidade.

4.28 Classe RemocaoUsuarioService

A classe RemocaoUsuarioService é responsável pela remoção completa de usuários do sistema. Seus principais atributos são DadosJackut, SessaoService e ComunidadeService.

Ela possui método público para remover usuário e métodos privados para remover comunidades do usuário, remover referências em outros usuários e remover sessões associadas.

Essa classe é importante porque remoção de conta afeta várias partes do sistema. Ao remover um usuário, o sistema precisa apagar suas comunidades, removê-lo de comunidades de outros usuários, remover amigos, convites, fãs, ídolos, paqueras, inimigos, recados e mensagens enviadas por ele. Centralizar essa regra em um serviço próprio evita espalhar a lógica de remoção por várias classes.

4.29 Classe ServicoPersistencia

A classe ServicoPersistencia implementa Persistidor e é responsável por salvar, carregar e limpar os dados persistidos do sistema.

Ela utiliza serialização Java por meio de ObjectOutputStream e ObjectInputStream. Seu arquivo padrão é dados-jackut.ser. Também possui métodos auxiliares privados para conversão de objeto carregado e fechamento de fluxos.

A existência dessa classe isola entrada e saída de arquivo. Sem ela, os serviços de negócio precisariam conhecer detalhes de serialização, o que violaria a separação entre regra de negócio e infraestrutura.

4.30 Classe FormatadorLista

A classe FormatadorLista é uma classe utilitária responsável por formatar listas no padrão esperado pelos testes de aceitação.

Seu principal método é formatar, que recebe uma lista de strings e retorna uma representação textual no formato {item1,item2,item3}.

Essa classe evita repetição de código de formatação em serviços diferentes. AmizadeService, ComunidadeService, IdoloService e PaqueraService podem reutilizar a mesma lógica, mantendo consistência no formato de saída.

4.31 Classe JackutException

A classe JackutException é a exceção base do domínio do sistema. Ela herda de RuntimeException e serve como superclasse para todas as exceções específicas do Jackut.

Além de armazenar a mensagem de erro, ela possui o método ajustarTexto, responsável por adaptar textos com acentuação à codificação padrão da JVM. Essa adaptação é relevante porque os testes do EasyAccept podem ser executados em ambientes com UTF-8 ou ISO-8859-1.

A existência dessa classe permite organizar uma hierarquia de exceções sem perder compatibilidade com os testes. Ela representa o erro genérico de domínio, enquanto as subclasses representam situações específicas.

4.32 Classe LoginInvalidoException

A classe LoginInvalidoException representa erro de login inválido no cadastro de usuário.

Ela é lançada quando o login informado não atende às regras exigidas pelo sistema. Sua existência evita o uso de uma exceção genérica e deixa claro que o erro ocorre especificamente na validação do login.

4.33 Classe SenhaInvalidaException

A classe SenhaInvalidaException representa erro de senha inválida no cadastro de usuário.

Ela é usada quando a senha informada é nula ou vazia, conforme as regras dos testes. Sua existência melhora a semântica do tratamento de erro e torna o código de UsuarioService mais expressivo.

4.34 Classe LoginOuSenhaInvalidosException

A classe LoginOuSenhaInvalidosException representa erro de autenticação.

Ela é lançada quando o usuário tenta abrir sessão com login inexistente ou senha incorreta. Essa exceção separa erros de cadastro de erros de autenticação, o que melhora a clareza das regras de SessaoService.

4.35 Classe UsuarioNaoCadastradoException

A classe UsuarioNaoCadastradoException representa a tentativa de acessar um usuário inexistente ou uma sessão inválida que não permite identificar um usuário cadastrado.

Ela é usada por vários serviços, como UsuarioService, SessaoService, PerfilService, AmizadeService e RemocaoUsuarioService. Sua mensagem padronizada mantém compatibilidade com os testes e evita duplicação de strings de erro.

4.36 Classe ContaJaExistenteException

A classe ContaJaExistenteException representa a tentativa de cadastrar uma conta com login já existente.

Ela é lançada por UsuarioService durante a criação de usuário. Sua existência deixa explícita a regra de unicidade do login no sistema.

4.37 Classe AtributoNaoPreenchidoException

A classe AtributoNaoPreenchidoException representa a tentativa de consultar um atributo de perfil que não foi preenchido.

Ela é usada por PerfilService. Essa exceção separa claramente erro de ausência de usuário de erro de ausência de atributo.

4.38 Classe UsuarioNaoPodeAdicionarASiMesmoException

A classe UsuarioNaoPodeAdicionarASiMesmoException representa a tentativa de um usuário adicionar a si mesmo como amigo.

Ela é lançada por AmizadeService. Sua existência deixa a regra de auto relacionamento explícita e evita mensagens genéricas.

4.39 Classe AmizadeJaExistenteException

A classe AmizadeJaExistenteException representa a tentativa de adicionar como amigo alguém que já é amigo.

Ela é usada por AmizadeService. Essa exceção expressa uma regra própria do relacionamento de amizade.

4.40 Classe ConviteAmizadePendenteException

A classe ConviteAmizadePendenteException representa a tentativa de enviar novamente um convite de amizade já pendente.

Ela é importante porque o fluxo de amizade do Jackut possui dois estados: convite pendente e amizade efetivada. Essa exceção diferencia esses estados de forma clara.

4.41 Classe UsuarioNaoPodeEnviarRecadoParaSiMesmoException

A classe UsuarioNaoPodeEnviarRecadoParaSiMesmoException representa a tentativa de enviar um recado para o próprio usuário.

Ela é lançada por RecadoService e impede uma operação não permitida pelos testes de aceitação.

4.42 Classe SemRecadosException

A classe SemRecadosException representa a tentativa de ler recado quando a caixa de entrada não possui recados pendentes.

Ela é usada por RecadoService. Sua existência torna explícito o estado de caixa de recados vazia.

4.43 Classe ComunidadeJaExistenteException

A classe ComunidadeJaExistenteException representa a tentativa de criar uma comunidade com nome já existente.

Ela é lançada por ComunidadeService. Como o nome da comunidade funciona como chave de identificação, essa exceção expressa a regra de unicidade das comunidades.

4.44 Classe ComunidadeNaoExisteException

A classe ComunidadeNaoExisteException representa a tentativa de acessar uma comunidade inexistente.

Ela é usada por ComunidadeService e por fluxos que dependem de comunidade válida, como envio de mensagens. Sua existência separa erros de comunidade de erros de usuário.

4.45 Classe UsuarioJaFazParteDaComunidadeException

A classe UsuarioJaFazParteDaComunidadeException representa a tentativa de adicionar um usuário a uma comunidade da qual ele já participa.

Ela é lançada por ComunidadeService. Essa exceção expressa a regra de não duplicidade de participação.

4.46 Classe SemMensagensException

A classe SemMensagensException representa a tentativa de ler mensagem de comunidade quando não há mensagens pendentes.

Ela é usada por MensagemService. Sua existência é semelhante a SemRecadosException, mas aplicada ao fluxo específico de mensagens comunitárias.

4.47 Classe IdoloJaExistenteException

A classe IdoloJaExistenteException representa a tentativa de adicionar novamente o mesmo ídolo.

Ela é lançada por IdoloService e expressa a regra de não duplicidade na relação fã e ídolo.

4.48 Classe PaqueraJaExistenteException

A classe PaqueraJaExistenteException representa a tentativa de adicionar novamente a mesma paquera.

Ela é usada por PaqueraService. Essa exceção separa erro de duplicidade de paquera de outros erros de relacionamento.

4.49 Classe InimigoJaExistenteException

A classe InimigoJaExistenteException representa a tentativa de adicionar novamente o mesmo inimigo.

Ela é usada por InimigoService e expressa a regra de não duplicidade em relações de inimizade.

4.50 Classe UsuarioNaoPodeSerFaDeSiMesmoException

A classe UsuarioNaoPodeSerFaDeSiMesmoException representa a tentativa de um usuário adicionar a si mesmo como ídolo.

Ela é lançada por IdoloService. Sua existência explicita a regra de que um usuário não pode ser fã de si mesmo.

4.51 Classe UsuarioNaoPodeSerPaqueraDeSiMesmoException

A classe UsuarioNaoPodeSerPaqueraDeSiMesmoException representa a tentativa de um usuário adicionar a si mesmo como paquera.

Ela é usada por PaqueraService e separa essa regra das demais validações de relacionamento.

4.52 Classe UsuarioNaoPodeSerInimigoDeSiMesmoException

A classe UsuarioNaoPodeSerInimigoDeSiMesmoException representa a tentativa de um usuário adicionar a si mesmo como inimigo.

Ela é lançada por InimigoService. Sua existência torna explícita a regra de auto inimizade inválida.

4.53 Classe FuncaoInvalidaException

A classe FuncaoInvalidaException representa uma operação bloqueada por relação de inimizade.

Ela recebe o nome do usuário alvo para compor a mensagem exigida pelos testes. É usada por BloqueioRelacionamentoService quando um usuário tenta realizar uma ação sobre alguém que o marcou como inimigo.

Essa exceção é importante porque a regra de inimizade não apenas registra uma relação, mas também bloqueia funções como adicionar amigo, adicionar ídolo, adicionar paquera e enviar recado.

4.54 Classe ErroPersistenciaException

A classe `ErroPersistenciaException` representa erros de leitura, gravação ou limpeza do arquivo de persistência.

Ela encapsula problemas de infraestrutura relacionados à serialização e evita que exceções técnicas sejam propagadas diretamente para as camadas de negócio. Essa decisão melhora o encapsulamento de falhas e torna a persistência mais coesa.

5. Interações entre os componentes

O funcionamento do sistema segue um fluxo em camadas. Os testes de aceitação interagem com a classe `Facade`. A `Facade` encaminha a operação para um serviço especializado. O serviço valida as regras de negócio, manipula modelos e, quando necessário, acessa `DadosJackut`. A persistência é feita por `ServicoPersistencia` por meio do contrato `Persistidor`.

De forma resumida, o fluxo principal pode ser descrito assim:

Testes `EasyAccept` → `Facade` → Serviços especializados → Modelos / `DadosJackut` → Persistência

Quando um teste chama `criarUsuario`, a `Facade` delega a chamada para `UsuarioService`. O serviço valida o login, valida a senha, verifica se já existe uma conta com o mesmo login e cria um objeto `Usuario`. Esse objeto é armazenado no mapa de usuários de `DadosJackut`. A `Facade` não conhece os detalhes dessa validação, apenas encaminha a operação.

Quando um teste chama `abrirSessao`, a `Facade` encaminha a chamada para `SessaoService`. O serviço busca o usuário por meio de `UsuarioService`, confere a senha e cria um objeto `Sessao`. O identificador da sessão é gerado por `DadosJackut` e a sessão é armazenada no mapa de sessões. Esse fluxo separa autenticação de cadastro.

Quando um teste chama `editarPerfil`, a `Facade` delega a operação para `PerfilService`. O serviço usa `SessaoService` para descobrir o usuário associado à sessão e, em seguida, chama `editarPerfil` em `Usuario`. O usuário delega a alteração para `PerfilUsuario`. Esse fluxo mostra a aplicação de composição: o serviço coordena a regra, `Usuario` representa a entidade e `PerfilUsuario` mantém os dados específicos do perfil.

Quando um teste chama `adicionarAmigo`, a `Facade` encaminha a operação para `AmizadeService`. O serviço valida a sessão do solicitante, busca o usuário alvo, verifica se existe bloqueio por inimizade, impede autoamizade, verifica amizade já existente e convite pendente. Caso o outro usuário já tenha enviado convite para o solicitante, a amizade é efetivada para ambos. Caso contrário, um convite fica pendente. Esse fluxo envolve `UsuarioService`, `SessaoService`, `BloqueioRelacionamentoService` e os métodos de `AmizadesUsuario` acessados por `Usuario`.

Quando um teste chama `enviarRecado`, a `Facade` delega para `RecadoService`. O serviço valida a sessão, busca o destinatário, verifica bloqueio por inimizade, impede envio para si mesmo e cria um `Recado`. Esse recado é entregue à `CaixaEntrada` do destinatário. Quando o teste chama `lerRecado`, `RecadoService` verifica se há recados pendentes e remove o primeiro da fila.

Quando um teste chama `criarComunidade`, a `Facade` delega para `ComunidadeService`. O serviço identifica o usuário dono pela sessão, verifica se já existe comunidade com o mesmo nome, cria a `Comunidade` e registra o dono como membro. Também adiciona o nome da comunidade em `ParticipacoesUsuario` do dono. Essa interação mostra como a criação de comunidade afeta tanto o estado global de `DadosJackut` quanto o estado individual de `Usuario`.

Quando um teste chama `adicionarComunidade`, `ComunidadeService` busca a comunidade e o usuário da sessão. Se o usuário já participa, uma exceção específica é lançada. Caso contrário, o usuário é adicionado à lista de membros da comunidade e a comunidade é registrada nas participações do usuário.

Quando um teste chama `enviarMensagem`, a `Facade` delega para `MensagemService`. O serviço valida a sessão do remetente, busca a comunidade por `ComunidadeService` e cria uma `MensagemComunidade`. Em seguida, percorre os membros da comunidade e entrega a mensagem para a `CaixaEntrada` de cada membro. Diferentemente de um recado, que possui um destinatário único, a mensagem de comunidade é distribuída para todos os membros.

Quando um teste chama `adicionarPaquera`, a `Facade` delega para `PaqueraService`. O serviço valida sessão, usuário alvo e bloqueio por inimizade. Em seguida, verifica se a paquera já existe e impede auto relacionamento. Caso o outro usuário também tenha adicionado o primeiro como paquera, `PaqueraService` cria recados automáticos para ambos. Essa regra fica isolada nesse serviço porque pertence apenas ao comportamento de paquera.

Quando um teste chama `adicionarInimigo`, a `Facade` delega para `InimigoService`. Esse serviço registra a relação de inimizade. A consequência dessa relação aparece em outros serviços por meio de `BloqueioRelacionamentoService`. Assim, a inimizade não apenas fica armazenada, mas também influencia operações futuras.

Quando um teste chama `removerUsuario`, a `Facade` delega para `RemocaoUsuarioService`. Esse serviço identifica o usuário da sessão, remove comunidades criadas por ele, remove sua participação em comunidades, apaga referências em outros usuários, remove mensagens e recados enviados por ele e exclui suas sessões. Esse fluxo é um dos mais complexos do projeto e justifica a existência de um serviço próprio.

Quando o teste chama `encerrarSistema`, a `Facade` usa `Persistidor` para salvar `DadosJackut`. A implementação concreta `ServicoPersistencia` grava o objeto em arquivo serializado. Ao iniciar o sistema novamente, a `Facade` carrega os dados

por meio do mesmo contrato. As sessões são reiniciadas, pois não representam estado persistente do domínio.

6. Conceitos de Programação Orientada a Objetos aplicados

6.1 Encapsulamento

O encapsulamento é aplicado em todas as classes principais do projeto. Os atributos são privados e o acesso ao estado ocorre por métodos específicos.

A classe `Usuario`, por exemplo, não expõe diretamente seus objetos internos de perfil, amizades, participações, relacionamentos e caixa de entrada. Métodos como `getAmigos`, `getComunidades`, `getFas` e `getPaqueras` retornam listas obtidas de objetos auxiliares, que por sua vez retornam cópias das listas internas. Isso impede que serviços externos modifiquem diretamente as estruturas internas sem passar pelas regras de negócio.

A classe `Comunidade` também aplica encapsulamento ao manter membros como lista privada e retornar uma cópia em `getMembros`. Assim, a alteração de membros precisa ocorrer por `adicionarMembro` ou `removerMembro`.

6.2 Herança

A herança é aplicada de forma objetiva na hierarquia de mensagens e na hierarquia de exceções.

A classe `Mensagem` representa uma abstração comum para objetos que possuem remetente e texto. `Recado` e `MensagemComunidade` herdam de `Mensagem`, reutilizando os atributos e métodos comuns.

A classe `JackutException` herda de `RuntimeException` e serve como base para todas as exceções específicas do domínio, como `LoginInvalidoException`, `ComunidadeNaoExisteException` e `SemMensagensException`.

A versão final evita herança artificial entre `Usuario` e `Sessao`. Em vez de herdar de uma classe `Entidade` apenas para compartilhar identificador, essas classes implementam a interface `Identificavel`. Essa decisão torna o design mais coerente, pois usa herança apenas quando existe uma relação conceitual forte.

6.3 Classe abstrata

A classe `Mensagem` é abstrata porque representa um conceito geral do sistema, mas não deve ser instanciada diretamente. O Jackut trabalha com tipos concretos de mensagem: `Recado` e `MensagemComunidade`.

Essa decisão evita duplicação de código e expressa corretamente o domínio. Todo recado e toda mensagem de comunidade possuem remetente e texto, mas uma mensagem genérica não tem função direta no sistema.

6.4 Polimorfismo

O polimorfismo aparece no uso de interfaces e na hierarquia de mensagens e exceções.

A interface `Persistidor` permite que a `Facade` trabalhe com uma referência abstrata de persistência. A implementação atual é `ServicoPersistencia`, mas outra implementação poderia ser usada no futuro sem alterar o contrato público da `Facade`.

A interface `Identificavel` permite que `Usuario` e `Sessao` sejam tratados como objetos identificáveis. Embora cada classe tenha natureza diferente, ambas cumprem o contrato `getId`.

Na hierarquia de mensagens, `Recado` e `MensagemComunidade` podem ser compreendidos como especializações de `Mensagem`. Na hierarquia de exceções, várias exceções específicas podem ser tratadas como `JackutException` quando se deseja lidar genericamente com erros de domínio.

6.5 Composição

A composição é um dos conceitos mais importantes da versão final. A classe `Usuario` é composta por `PerfilUsuario`, `AmizadesUsuario`, `ParticipacoesUsuario`, `RelacionamentosUsuario` e `CaixaEntrada`.

Essa decisão reduz a complexidade de `Usuario` e atribui responsabilidades aos objetos que possuem os dados adequados. `PerfilUsuario` conhece os atributos de perfil. `AmizadesUsuario` conhece amigos e convites. `ParticipacoesUsuario` conhece comunidades. `RelacionamentosUsuario` conhece ídolos, fãs, paqueras e inimigos. `CaixaEntrada` conhece recados e mensagens.

Também existe composição entre `CaixaEntrada` e seus recados e mensagens, pois a caixa de entrada mantém as filas de objetos recebidos pelo usuário.

6.6 Uso de listas

O sistema utiliza listas para representar coleções ordenadas de elementos do domínio.

`AmizadesUsuario` usa listas para amigos e convites enviados. `ParticipacoesUsuario` usa lista para comunidades. `RelacionamentosUsuario` usa listas para ídolos, fãs, paqueras e inimigos. `Comunidade` usa lista para membros. `CaixaEntrada` usa `LinkedList` para recados e mensagens, pois esses elementos precisam ser lidos em ordem de chegada.

O uso de listas é acompanhado de encapsulamento. Os métodos de consulta retornam cópias, evitando exposição direta das listas internas.

6.7 Tratamento de exceções

O tratamento de exceções é realizado por uma hierarquia específica do domínio. JackutException é a classe base, e cada erro relevante possui uma subclasse própria.

Essa abordagem melhora a semântica do código. Em vez de lançar sempre uma exceção genérica com mensagens diferentes, o sistema expressa o tipo do erro por meio da classe lançada. Por exemplo, ComunidadeNaoExisteException indica erro em comunidade, SemRecadosException indica ausência de recados e FuncaoInvalidaException indica operação bloqueada por inimizade.

O uso de RuntimeException é adequado ao contexto do EasyAccept, pois os métodos públicos da Facade não precisam declarar todas as exceções verificadas. As mensagens permanecem compatíveis com os testes de aceitação.

6.8 Interface

O projeto usa interfaces para representar contratos de comportamento.

Persistidor define o contrato de persistência, permitindo que ServicoPersistencia seja substituído por outra implementação no futuro. Identificavel define o contrato de objetos identificáveis, implementado por Usuario e Sessao.

O uso de interfaces reduz acoplamento e evita dependência desnecessária de implementações concretas. Isso está alinhado ao princípio de programar para uma interface, e não para uma implementação.

6.9 Abstração

A abstração aparece na separação entre conceitos gerais e concretos. Mensagem representa o conceito abstrato de uma mensagem com remetente e texto. Recado e MensagemComunidade representam formas concretas desse conceito.

Persistidor abstrai a ideia de persistência sem expor a forma como os dados são gravados. Facade abstrai a complexidade interna do sistema para os testes de aceitação. Usuario abstrai o conceito de pessoa cadastrada, enquanto seus objetos internos detalham aspectos específicos do estado social.

6.10 Responsabilidade única

A versão final aplica o princípio da responsabilidade única de forma mais forte que a versão inicial.

SistemaJackut deixa de existir porque concentrava muitas responsabilidades. Em seu lugar, surgem serviços com responsabilidades específicas. UsuarioService trata cadastro e busca. SessaoService trata autenticação e sessão. ComunidadeService trata comunidades. MensagemService trata mensagens de comunidade. PaqueraService trata paqueras. RemocaoUsuarioService trata remoção de contas.

Essa divisão torna o código mais fácil de alterar. Uma mudança em regra de paquera tende a afetar PaqueraService, e não todas as funcionalidades do sistema.

6.11 Coesão e acoplamento

A coesão melhora porque cada classe possui uma finalidade clara. As classes de serviço agrupam regras relacionadas. As classes de modelo representam dados e comportamentos próprios do domínio. As exceções representam erros específicos. Os contratos definem abstrações.

O acoplamento é reduzido por meio de interfaces, delegação e serviços especializados. A Facade depende dos serviços, mas os testes não dependem da estrutura interna. Os serviços dependem de UsuarioService e SessaoService para operações comuns, evitando duplicação de busca e validação.

Ainda existe acoplamento natural entre os serviços e o domínio, pois as regras de negócio precisam manipular Usuario, Comunidade, Recado e MensagemComunidade. Esse acoplamento é aceitável porque corresponde às relações reais do problema.

6.12 Serialização

A serialização é aplicada para persistir DadosJackut em arquivo. As classes de modelo que fazem parte do estado persistível implementam Serializable.

Essa escolha atende ao escopo do projeto e aos testes de persistência. A desvantagem é que arquivos serializados não são legíveis diretamente por humanos, ao contrário de formatos como XML ou JSON. Entretanto, para o contexto do projeto e sem uso de frameworks externos, a serialização Java é uma solução simples e suficiente.

7. Padrões de projeto e princípios arquiteturais adotados

7.1 Nome do padrão: Facade

O padrão Facade fornece uma interface simplificada para um conjunto de classes internas mais complexo.

No Jackut, o problema resolvido é a necessidade de expor uma única classe para os testes de aceitação sem revelar a organização interna do sistema. Os testes chamam métodos da classe Facade, mas não conhecem UsuarioService, SessaoService, ComunidadeService ou qualquer classe de modelo.

A oportunidade de aplicação surge pela própria estrutura do projeto, que exige uma classe Facade compatível com o EasyAccept. A versão final aplica esse padrão de forma mais adequada que a versão inicial, porque a Facade não

delega tudo para uma segunda fachada. Ela encaminha diretamente para serviços especializados.

No projeto, a Facade expõe métodos como `criarUsuario`, `abrirSessao`, `adicionarAmigo`, `enviarRecado`, `criarComunidade`, `enviarMensagem` e `removerUsuario`. Internamente, cada método chama o serviço responsável pela regra. Essa decisão preserva o contrato público e permite alterar a estrutura interna sem afetar os testes.

7.2 Nome do padrão: Strategy aplicado à persistência

O padrão Strategy permite variar uma implementação por meio de um contrato comum.

No Jackut, o problema resolvido é o acoplamento entre o sistema e a forma concreta de persistência. Se a Facade dependesse diretamente de `ServicoPersistencia`, qualquer mudança de persistência exigiria alteração no ponto de entrada do sistema.

A oportunidade de aplicação surge na necessidade de salvar, carregar e limpar dados. Essas operações são definidas pela interface `Persistidor`. A classe `ServicoPersistencia` implementa esse contrato usando serialização Java.

No projeto, a Facade possui um atributo do tipo `Persistidor`. A implementação concreta usada é `ServicoPersistencia`. Essa estrutura permite que uma futura implementação, como `PersistidorXml` ou `PersistidorBancoDados`, substitua a atual desde que respeite o mesmo contrato.

7.3 Nome do padrão arquitetural: Service Layer

O padrão Service Layer organiza a lógica de negócio em serviços que ficam entre a camada de entrada e os modelos de domínio.

No Jackut, o problema resolvido é a concentração de regras em uma única classe. A versão inicial concentrava muitas responsabilidades em `SistemaJackut`. Com o crescimento do projeto no Milestone 02, essa abordagem dificultaria manutenção e extensão.

A oportunidade de aplicação surge com a adição de comunidades, mensagens, novos relacionamentos e remoção de conta. Cada conjunto de regras passa a ter um serviço próprio.

No projeto, `UsuarioService`, `SessaoService`, `PerfilService`, `AmizadeService`, `RecadoService`, `ComunidadeService`, `MensagemService`, `IdoloService`, `PaqueraService`, `InimigoService`, `BloqueioRelacionamentoService` e `RemocaoUsuarioService` formam uma camada de serviços. Essa camada concentra regras de negócio sem poluir a Facade nem sobrecarregar os modelos.

7.4 Nome do princípio: Expert

O princípio Expert recomenda atribuir uma responsabilidade à classe que possui as informações necessárias para cumpri-la.

No Jackut, esse princípio aparece em várias decisões. PerfilUsuario guarda e manipula atributos de perfil porque possui o mapa de atributos. CaixaEntrada recebe e remove recados porque possui as filas. Comunidade adiciona e remove membros porque possui a lista de membros. BloqueioRelacionamentoService valida bloqueios porque centraliza a regra de inimizade usada por diferentes serviços.

A oportunidade de aplicação surge durante a refatoração de Usuario e SistemaJackut. Em vez de manter todas as listas e regras no mesmo lugar, as responsabilidades são distribuídas entre classes que possuem os dados adequados.

Essa decisão melhora a coesão e facilita futuras extensões. Se o formato da caixa de entrada mudar, a alteração tende a ficar em CaixaEntrada. Se a regra de perfil mudar, a alteração tende a ficar em PerfilUsuario ou PerfilService.

8. Persistência de dados

A persistência do sistema é implementada por meio de serialização Java. A classe ServicoPersistencia salva e carrega um objeto DadosJackut no arquivo dados-jackut.ser.

DadosJackut centraliza os mapas de usuários, sessões e comunidades. Os usuários armazenam, por composição, seus perfis, amizades, participações, relacionamentos, recados e mensagens. Assim, ao salvar DadosJackut, o sistema preserva o estado necessário para os testes de persistência.

Embora DadosJackut possua mapa de sessões, as sessões são reiniciadas após o carregamento. Essa decisão é adequada porque sessões representam estado temporário de execução. Em uma nova execução do sistema, o usuário deve abrir uma nova sessão.

A persistência também contempla os novos elementos do Milestone 02, como comunidades, membros de comunidades, mensagens pendentes, relações de fã, ídolo, paquera, inimigo e remoção de usuários. Como todos esses dados estão ligados a DadosJackut ou aos objetos Usuario, eles são preservados pela serialização.

A vantagem dessa abordagem é a simplicidade. O projeto não depende de banco de dados, bibliotecas externas ou configuração adicional. A desvantagem é que o arquivo serializado não é legível diretamente por humanos. Para o escopo da disciplina e dos testes de aceitação, essa solução é adequada.

9. Compatibilidade de execução

O projeto considera diferenças de codificação entre ambientes de execução. Os testes do EasyAccept podem utilizar ISO-8859-1, enquanto ambientes como IntelliJ IDEA podem executar a JVM em UTF-8.

Para evitar falhas em comparações de mensagens com acentuação, a classe JackutException possui o método ajustarTexto. Esse método adapta mensagens ao charset padrão da JVM. A mesma estratégia é usada em mensagens automáticas de paquera, que também possuem acentuação.

Essa compatibilidade é importante porque o projeto pode ser executado em máquinas diferentes, com configurações distintas de codificação. A adaptação evita que o sistema passe em um ambiente e falhe em outro apenas por diferença de encoding.

A classe Main também contribui para compatibilidade ao localizar automaticamente a pasta tests. Dessa forma, o projeto não depende de caminhos absolutos específicos do computador de desenvolvimento.

va não permite inspeção direta do arquivo gerado. Além disso, como o projeto é executado em Java puro e guiado por testes de aceitação, algumas decisões permanecem ajustadas ao contrato esperado pelos testes, especialmente mensagens de erro e formatos de listas. Ainda assim, essas limitações são aceitáveis para o escopo da disciplina.

A versão final apresenta uma arquitetura mais coesa, menos acoplada e mais extensível. As decisões de design favorecem manutenção, clareza e evolução futura do sistema, mantendo compatibilidade com os testes de aceitação e com diferentes ambientes de execução.

10. Testes executados

A execução dos testes de aceitação do EasyAccept não apresentou falhas. Como os testes são escritos em arquivos de script, cada linha executável dos arquivos abaixo foi considerada um teste individual validado pela ferramenta.

Total geral: 473 testes OK, 0 testes FAILED.

Todos os 473 testes foram executados com sucesso.

11. Conclusão

O projeto Jackut, em sua versão final, entrega as funcionalidades previstas para os Milestones 01 e 02. O sistema implementa criação de contas, autenticação por sessões, consulta e edição de perfil, convites e efetivação de amizades, envio e leitura de recados, criação e participação em comunidades, envio e leitura de mensagens de comunidade, relações de fã e ídolo, paqueras, inimigos e remoção completa de usuários.

A versão final também apresenta uma evolução arquitetural significativa em relação ao Milestone 01. A classe SistemaJackut deixa de existir e suas responsabilidades são distribuídas entre serviços especializados. Essa alteração corrige a concentração excessiva de responsabilidades em uma única classe e melhora a aderência ao princípio da responsabilidade única. A Facade permanece como ponto de entrada dos testes de aceitação, mas passa a atuar apenas como camada de acesso, delegando as operações para serviços de negócio como UsuarioService, SessaoService, ComunidadeService, MensagemService, IdoloService, PaqueraService, InimigoService e RemocaoUsuarioService.

Os conceitos de orientação a objetos são aplicados de forma mais consistente na versão final. O encapsulamento aparece na proteção das estruturas internas dos modelos, especialmente em Usuario, Comunidade, AmizadesUsuario, ParticipacoesUsuario e RelacionamentosUsuario. A composição é utilizada para dividir o estado do usuário em PerfilUsuario, AmizadesUsuario, ParticipacoesUsuario, RelacionamentosUsuario e CaixaEntrada. A herança aparece de forma mais apropriada na hierarquia Mensagem, Recado e MensagemComunidade, além da hierarquia de exceções baseada em JackutException. O polimorfismo aparece no uso das interfaces Persistidor e Identificavel, que permitem trabalhar com contratos em vez de depender apenas de implementações concretas.

O tratamento de exceções também evolui. Em vez de concentrar todos os erros em uma única exceção genérica, o sistema possui exceções específicas para regras de negócio distintas, como ComunidadeNaoExisteException, UsuarioJaFazParteDaComunidadeException, SemMensagensException, IdoloJaExistenteException, PaqueraJaExistenteException, InimigoJaExistenteException e FuncaoInvalidaException. Essa estrutura melhora a legibilidade das regras e torna o código mais expressivo.

A arquitetura atual favorece expansão futura porque separa as regras em serviços coesos. Novos tipos de relacionamento podem ser adicionados por meio de novos serviços sem alterar significativamente os serviços já existentes. Novas funcionalidades de comunidade podem evoluir a partir de ComunidadeService e MensagemService. Mudanças na forma de persistência podem ser realizadas por uma nova implementação da interface Persistidor. Dessa forma, a estrutura atual reduz o impacto de alterações futuras.

Como limitação, a persistência por serialização Java não permite inspeção direta dos dados salvos em arquivo, diferentemente de formatos como XML ou JSON. Além disso, algumas decisões permanecem fortemente condicionadas pelo contrato do EasyAccept, especialmente o formato exato das mensagens de erro, os nomes dos métodos públicos da Facade e a representação textual de listas. Essas limitações são aceitáveis no contexto do projeto, mas poderiam ser revistas em uma versão futura com uma camada de interface mais independente dos testes de aceitação.

De modo geral, a versão final do Jackut atende aos requisitos funcionais, preserva compatibilidade com os testes de aceitação e apresenta uma arquitetura mais coesa, menos acoplada e mais extensível que a versão inicial.

12. Avaliação Objetiva

A avaliação objetiva do projeto Jackut em sua versão final é a seguinte:

- **Qualidade da documentação – 9,0:** O projeto apresenta Javadocs nas principais classes e métodos, package-info.java nos pacotes e um relatório que descreve a arquitetura, os componentes, os conceitos de orientação a objetos, os padrões aplicados, a persistência e os testes executados. A documentação é clara e acompanha a evolução do Milestone 01 para o Milestone 02. A nota não é 10,0 porque a quantidade de classes torna o diagrama completo difícil de representar em uma única imagem, exigindo diagramas simplificados por visão arquitetural.
- **Qualidade do design arquitetural – 9,5:** A arquitetura apresenta boa separação de responsabilidades, com Facade fina, serviços especializados, modelos coesos e persistência isolada por contrato. A remoção de SistemaJackut corrige a característica de God Class existente na versão inicial, e a divisão em UsuarioService, SessaoService, ComunidadeService, MensagemService, IdoloService, PaqueraService, InimigoService, BloqueioRelacionamentoService e RemocaoUsuarioService melhora a coesão. A nota não é 10,0 porque ainda existe acoplamento natural entre serviços e modelos centrais, especialmente Usuario e DadosJackut, embora esse acoplamento seja aceitável para o domínio.
- **Qualidade do código – 9,5:** O código apresenta nomenclatura clara, organização por pacotes, métodos com responsabilidades bem definidas e ausência de regras de negócio na Facade. Classes como FormatadorLista, CaixaEntrada, PerfilUsuario e BloqueioRelacionamentoService reduzem repetição e concentram responsabilidades específicas. A nota não é 10,0 porque algumas regras ainda dependem de strings exigidas pelos testes, como mensagens de erro e nomes de atributos de perfil, o que reduz parte da flexibilidade.
- **Tratamento de exceções – 9,5:** O sistema possui uma hierarquia de exceções específica e semântica. JackutException atua como classe base, enquanto exceções como LoginInvalidoException, UsuarioNaoCadastradoException, ComunidadeNaoExisteException, SemRecadosException, SemMensagensException e FuncaoInvalidaException representam erros distintos do domínio. A nota não é 10,0 porque as mensagens ainda ficam acopladas ao contrato dos testes de aceitação, mas isso é necessário para compatibilidade com o EasyAccept.
- **Aplicação de conceitos de POO – 9,5:** O projeto aplica encapsulamento, abstração, herança, polimorfismo, composição, interfaces e uso de coleções de forma coerente. A composição em Usuario melhora a modelagem do domínio, a hierarquia Mensagem/Recado/MensagemComunidade representa herança com

propósito real, e Persistidor permite polimorfismo na persistência. A nota não é 10,0 porque o domínio do Jackut não exige uma hierarquia extensa de tipos de usuário ou comunidades, então parte do polimorfismo aparece de forma pontual e não como eixo central do sistema.